

Gradebook with Class Statistics

A C Programming Project

Submitted By: Harsh Vardhan

Guided By: Ms. Naina Devi

Course: Programming in C

Tool Used: VS Code

Agenda

- Introduction
- Problem Statement
- Tools and Technologies
- Project Description
- Project Working
- Features
- Applications
- Advantages
- Future Scope
- Conclusion

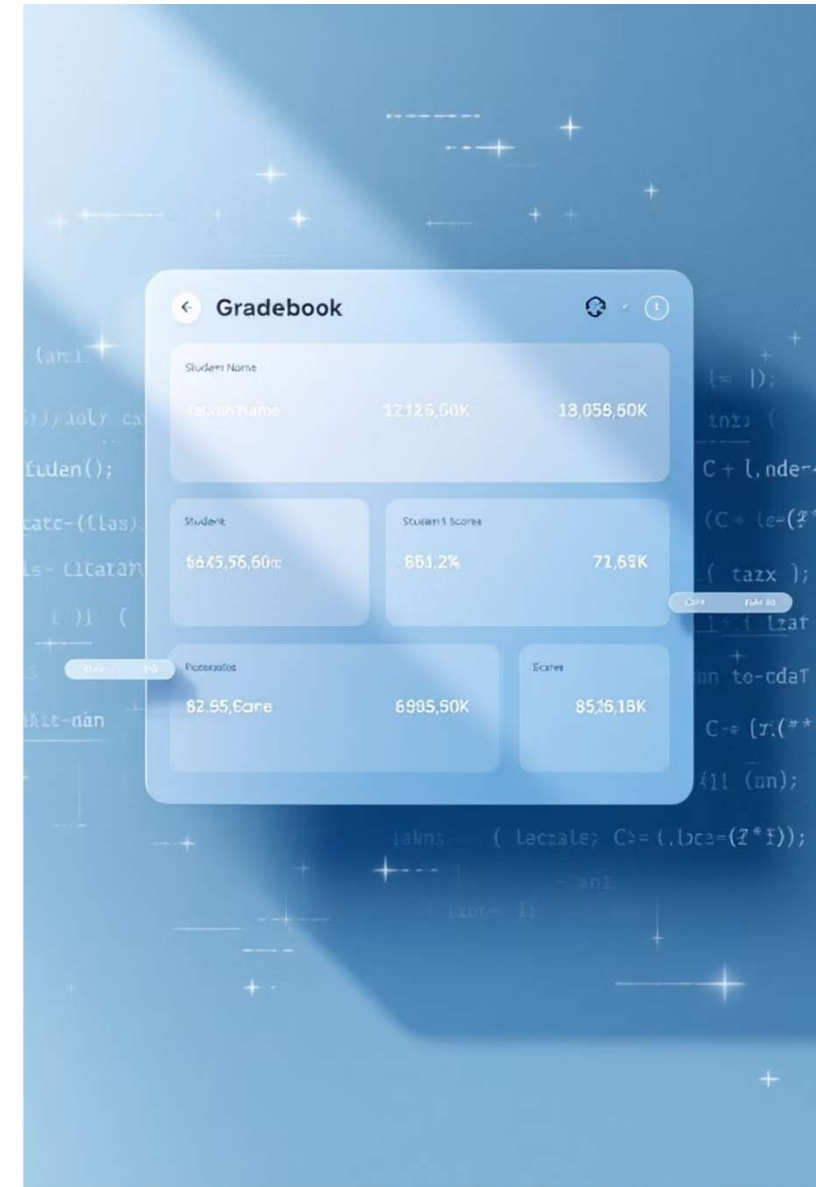
Introduction

The Gradebook with Class Statistics project is a simple C-based program designed to manage students' marks and analyze academic performance.

It helps in calculating:

- Average marks
- Top score
- Grade distribution

This project demonstrates the practical use of loops, conditional statements, and basic logic in C programming.



Problem Statement

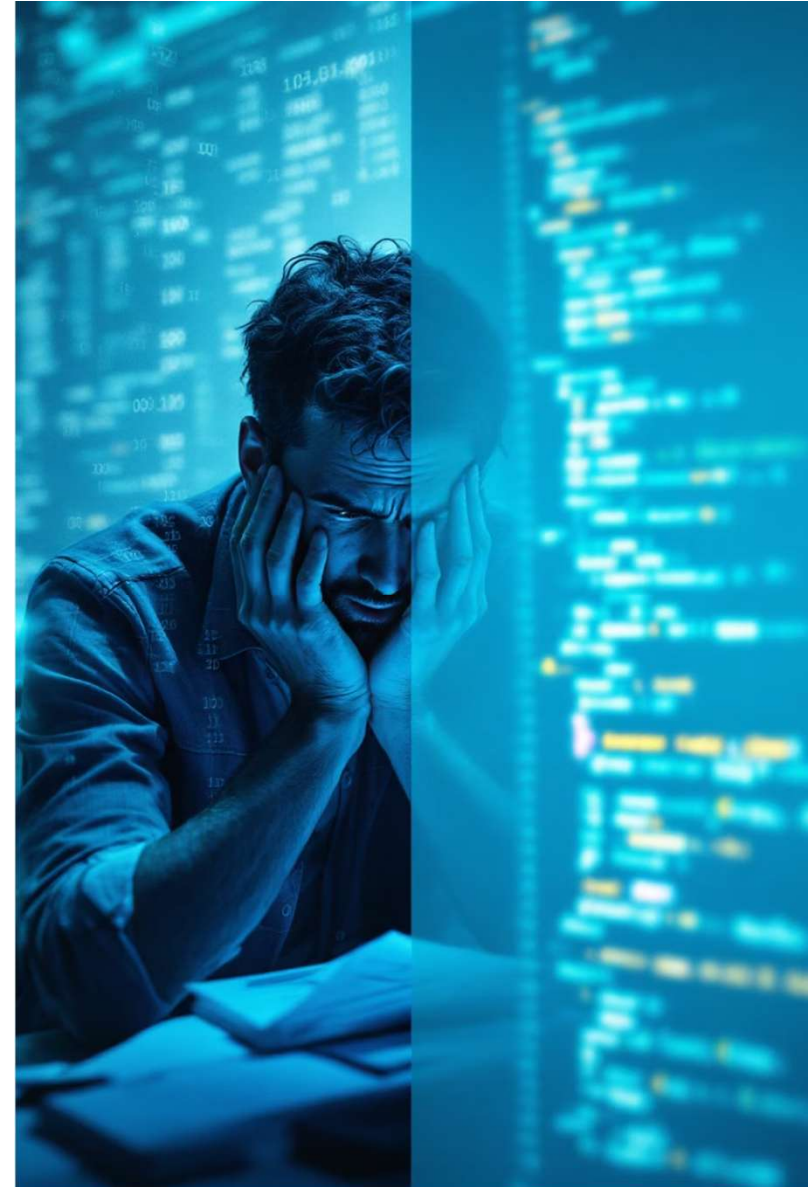
The Challenge: Manual Calculations

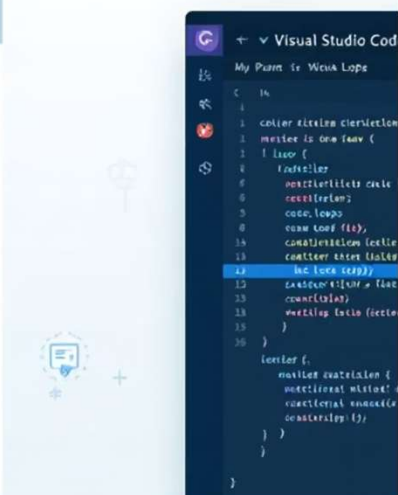
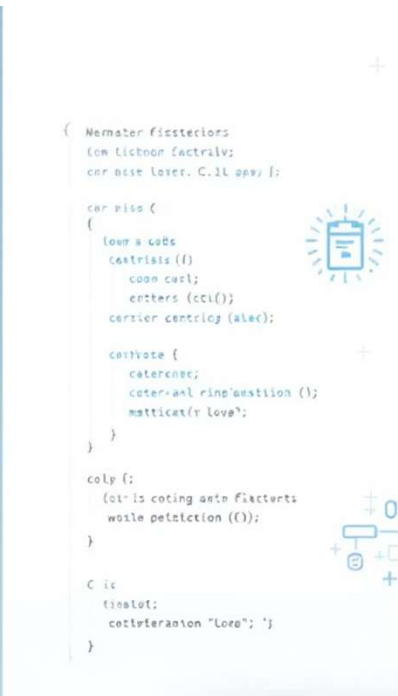
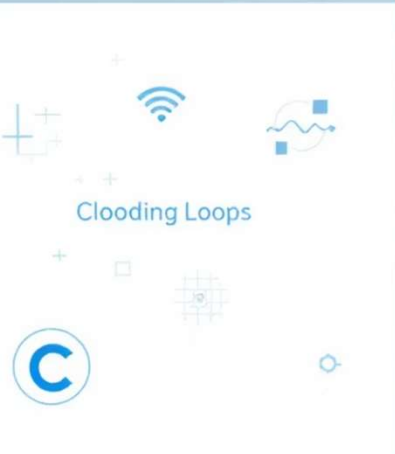
Manually calculating class statistics such as average marks, highest marks, and grade distribution is time-consuming and error-prone.

Our Solution: An Automated C Program

Develop a C program that:

- Takes marks of multiple students
- Automatically assigns grades
- Generates class statistics efficiently
- Works without using arrays (memory efficient)





Tools and Technologies

Programming Language:

C

IDE:

Visual Studio Code (VS Code)

Platform:

Windows

Concepts Used:

- Loops
- Conditional Statements
- Variables
- Basic Arithmetic Operations

Project Description

The program accepts the number of students and their marks one by one.

It processes data without using arrays and calculates:

Total and average marks

Highest marks in the class

Number of students in each grade
(A, B, C, D, F)

Grades are assigned based on predefined mark ranges.

```

1 #include <stdio.h>
2
3 int main() {
4     int n, i, marks;
5     int topScore = 0;
6     float sum = 0, average;
7
8     int gradeA = 0, gradeB = 0, gradeC = 0, gradeD = 0, gradeF = 0;
9
10    printf("Enter number of students: ");
11    scanf("%d", &n);
12
13    for(i = 1; i <= n; i++) {
14        printf("Enter marks of student %d: ", i);
15        scanf("%d", &marks);
16
17        sum += marks;
18
19        if(marks > topScore)
20            topScore = marks;
21
22        if(marks >= 90)
23            gradeA++;
24        else if(marks >= 80)
25            gradeB++;

```

```

26        else if(marks >= 70)
27            gradeC++;
28        else if(marks >= 60)
29            gradeD++;
30        else
31            gradeF++;
32    }
33
34    average = sum / n;
35
36    printf("\n----- Class Statistics ----- \n");
37    printf("Average Marks: %.2f \n", average);
38    printf("Top Score: %d \n", topScore);
39
40    printf("\nGrade Distribution: \n");
41    printf("A Grade: %d \n", gradeA);
42    printf("B Grade: %d \n", gradeB);
43    printf("C Grade: %d \n", gradeC);
44    printf("D Grade: %d \n", gradeD);
45    printf("F Grade: %d \n", gradeF);
46
47    return 0;
48 }

```

Project Working

User enters number of students

Marks are entered one by one

Program:

- Adds marks to total
- Finds top score
- Counts grades

Final class statistics are displayed



Applications

- Schools and Colleges
- Teacher's mark evaluation
- Academic performance analysis
- Learning project for C programming beginners
- Small-scale examination systems



Advantages

1

Simple and easy to use

2

No array used (low memory consumption)

3

Fast and accurate calculations

4

Beginner-friendly logic

5

Can be extended for advanced features

HOW TO CARREL TAGE Benefits on Advantages



Future Scope & Conclusion

FUTURE SCOPE:

- Add student names and roll numbers
- Store data using files
- Convert to menu-driven program
- Graphical grade distribution
- Web or GUI-based version

CONCLUSION:

This project successfully demonstrates how C programming can be used to analyze academic data efficiently. It strengthens core programming concepts and provides a foundation for advanced applications.